

Kinh nghiệm của những người giỏi nhất (What the Masters Actually Do)

PHỤ THUỘC A/01 — chương này nói cách luyện để quy trình ở chương 1 thành phản xạ. Đọc được độc lập, nhưng nhiều lời khuyên ở đây chỉ có nghĩa khi bạn đã biết bốn bước.

TÓM TẮT

Chương này khảo sát cách làm việc thật của những người đã định hình ngành thuật toán — Knuth, Dijkstra, Hoare, Tarjan, Bentley, Skiena, Sedgewick — cùng cách luyện của giới lập trình thi đấu hiện đại và của giới huấn luyện olympic toán. Mục đích không phải kể chuyện danh nhân mà là rút ra những thói quen *bắt buộc được*: viết chương trình cùng lúc với chứng minh, ước lượng trước khi cài, luyện ở vùng hơi quá sức, upsolving sau mỗi trận, giữ nhật ký lỗi, tự xây thư viện mẫu. Chương cũng cho một bản đồ sách: quyển nào đọc lúc nào và để làm gì. Nếu chỉ nhớ một điều: điểm chung của tất cả những người này không phải năng khiếu mà là *họ có một vòng phản hồi chặt* — làm, đo, phát hiện sai, sửa cách làm, lặp lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **luyện tập có chủ đích (deliberate practice)** — luyện có mục tiêu cụ thể, ở mức hơi quá sức hiện tại, có phản hồi ngay và có sửa sai — khác hẳn với “làm nhiều giờ”. Khái niệm của Ericsson[1].
- **upsolving** — giải lại những bài bạn không giải được trong một kỳ thi, *sau khi* thi xong, tự làm tới cùng. Đây là nguồn tiên bộ chính của dân thi đấu.
- **editorial** — bài phân tích lời giải chính thức của một đề thi lập trình.
- **chunk (khối tri thức)** — một cụm thông tin đã được ghép thành một đơn vị nhớ duy nhất, ví dụ “thế cờ này” thay vì hai mươi quân rời. Chuyên gia nhớ nhiều hơn không phải vì trí nhớ tốt hơn mà vì họ có nhiều khối lớn hơn.
- **thư viện mẫu (template library)** — tập hợp các đoạn mã đã kiểm chứng — DSU, segment tree, Dijkstra — được chuẩn bị sẵn để dán vào khi thi. KACTL của KTH là ví dụ nổi tiếng[2].
- **nhật ký lỗi** — sổ ghi lại mỗi lần sai: sai gì, vì sao, dấu hiệu nào lẽ ra đã phát hiện sớm. Công cụ đơn giản nhất và bị bỏ qua nhiều nhất.

CÂU HỎI MỞ ĐẦU

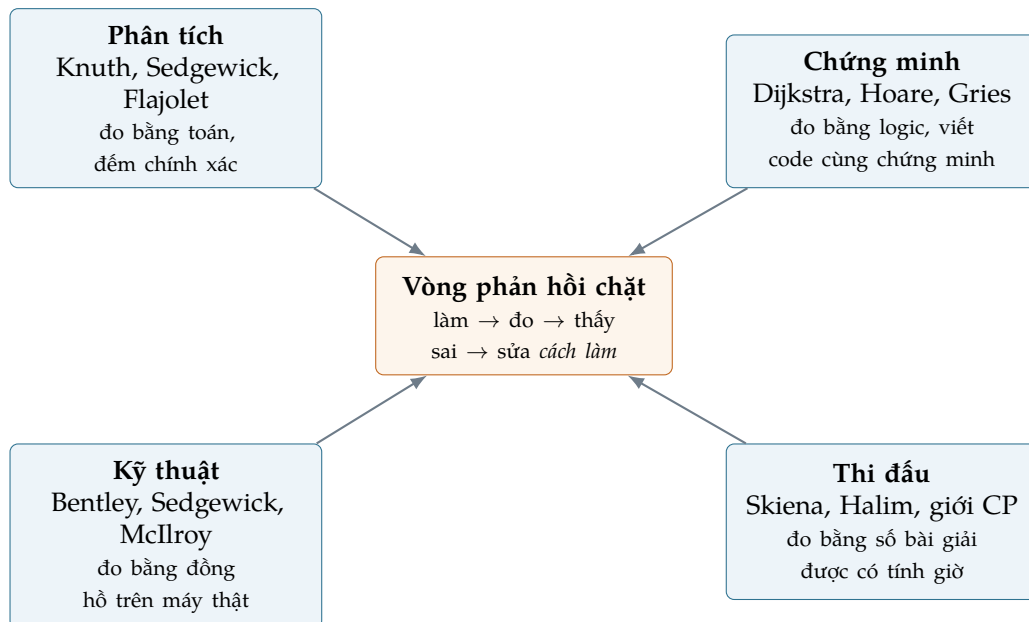
- Q1. Nếu tài năng là thứ quyết định, vì sao gần như mọi người giỏi thuật toán đều mô tả cùng một kiểu thói quen làm việc?
- Q2. Knuth ra bài tập có mức khó từ 0 đến 50 và bảo người đọc phải làm. Vì sao đọc mà không làm bài tập lại gần như vô ích?
- Q3. Dijkstra viết chương trình và chứng minh cùng lúc, không phải viết xong rồi kiểm thử. Điều đó thay đổi cái gì trong cách bạn ngồi trước bàn phím?
- Q4. Vì sao “giải thật nhiều bài” lại có thể khiến bạn dừng tiến bộ sau vài tháng, trong khi giải ít bài hơn nhưng chọn kỹ thì tiếp tục lên?
- Q5. Người thi đấu hàng đầu nói phần lớn tiến bộ đến từ upsolving chứ không từ chính kỳ thi. Cơ chế nào giải thích điều đó?

Có một cách đọc chương này sai ngay từ đầu, và đáng nói trước: đọc như đọc tiểu sử danh nhân, thấy hay, rồi không thay đổi gì. Vì vậy mỗi mục dưới đây được viết theo một khuôn cố định — người này làm gì cụ thể, vì sao nó hiệu quả, và bạn bắt chước được phần nào ngay tuần này. Những gì không bắt chước được, tôi sẽ nói thẳng là không bắt chước được.

Cũng cần nói rõ giới hạn: những phát biểu dưới đây dựa trên sách và tài liệu công khai của chính họ, không phải suy đoán về đời tư. Khi tôi mô tả thói quen của giới thi đấu, đó là những điều lặp đi lặp lại trong lời khuyên công khai của nhiều người giỏi, chứ không phải câu trích nguyên văn của một cá nhân cụ thể.

1.1 Bốn trường phái, một vòng phản hồi

Nếu xếp những người có ảnh hưởng nhất lên một sơ đồ, họ rơi vào bốn trường phái với bốn niềm tin khác nhau về việc “thế nào là làm thuật toán cho tốt”. Điều thú vị là bốn trường phái này không mâu thuẫn; chúng bổ sung nhau, và người mạnh thường lấy từ cả bốn.



Hình 1.1. Bốn trường phái khác nhau ở thứ họ dùng làm thước đo, nhưng giống nhau ở chỗ đều có một thước đo khách quan và đều sửa cách làm dựa trên kết quả đo. Người luyện một mình mà không có thước đo nào là người duy nhất không có vòng phản hồi — và đó là kiểu luyện phổ biến nhất.

Trường phái phân tích: Knuth và việc đếm cho chính xác

Donald Knuth xây cả sự nghiệp trên một niềm tin: muốn hiểu một thuật toán thì phải *đếm* được nó làm bao nhiêu phép toán, chứ không chỉ nói “cỡ $n \log n$ ”. Trong *The Art of Computer Programming* [3], [4], các thuật toán được phân tích tới hằng số, trên một máy ảo mà ông tự thiết kế để không phải phụ thuộc vào máy thật nào.

Hai thói quen của Knuth đáng bắt chước ngay. Thứ nhất là *bài tập có xếp hạng độ khó*: mỗi bài trong TAOCP có một số từ 0 đến 50, trong đó khoảng 20 là bài tập bình thường, còn 50 là bài toán mở nghiên cứu. Ý nghĩa của cách xếp hạng đó không phải để khoe độ khó mà để người đọc *chọn đúng vùng luyện* — ý tưởng mà mục sau sẽ nói kỹ. Thứ hai là thói quen coi việc phát hiện lỗi là có giá trị: Knuth nổi tiếng vì trả tiền thưởng cho mỗi lỗi được tìm thấy trong sách của ông. Thông điệp ẩn sau đó rất thực dụng: một người muốn đúng thì phải *tạo động lực cho việc tìm ra mình sai*, chứ không phòng thủ với nó.

Cũng của Knuth là câu bị trích sai nhiều nhất trong ngành: “tối ưu hoá sớm là gốc rễ của mọi tội lỗi”[5]. Nguyên văn nói rõ hơn nhiều — ý ông là trong phần lớn thời gian ta nên bỏ qua những cải thiện nhỏ, nhưng *không được* bỏ qua cơ hội ở phần mã thực sự quan trọng, và cách biết phần nào quan trọng là đo chứ không đoán. Đây chính là cầu nối sang trường phái kỹ thuật.

Trường phái chứng minh: Dijkstra, Hoare và kỷ luật viết code

Edsger Dijkstra có một quan điểm cực đoan và rất hữu ích: chương trình và chứng minh tính đúng của nó phải được phát triển *cùng lúc*, chứ không phải viết code trước rồi kiểm thử sau[6]. Ông cũng để lại câu nói chính xác về giới hạn của kiểm thử: kiểm thử cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt* của lỗi.

Nghe có vẻ hàn lâm, nhưng hệ quả thực hành rất cụ thể và rất rẻ: trước khi viết một vòng lặp, hãy viết ra — bằng lời, trong một dòng chú thích — *bất biến* mà vòng lặp đó duy trì. Chỉ một dòng đó thôi đã loại được phần lớn lỗi lệch một đơn vị và lỗi khởi tạo sai, vì mọi câu hỏi kiểu “chạy tới n hay n-1” đều có câu trả lời suy ra từ bất biến. Tony Hoare cấp nền hình thức cho cách làm này[7], còn David Gries biến nó thành giáo trình dạy được[8]. Chương A/04 lấy đúng ý này làm trung tâm.

Cần nói cả mặt trái. Chứng minh hình thức đầy đủ cho mọi chương trình là không thực tế trong công việc hằng ngày, và Dijkstra bị phê phán chính ở điểm đó. Cách dùng hợp lý là dùng *ý thức về bất biến* chứ không phải bộ máy hình thức: viết bất biến ra thành lời, và chỉ hình thức hoá ở những chỗ thật sự tinh vi.

Trường phái kỹ thuật: Bentley và thói quen đo

Jon Bentley viết *Programming Pearls*[9] như những chuyên mục ngắn, mỗi bài xoay quanh một bài toán thật gặp trong công việc. Ba thói quen của ông dùng được ngay.

Thứ nhất là *ước lượng phong bì* — tính nhẩm quy mô trước khi cài: bao nhiêu phần tử, mỗi phần tử bao nhiêu byte, một giây làm được cỡ bao nhiêu phép toán. Ước lượng thô loại được những hướng sai hằn với chi phí gần bằng không, và nó là kỹ năng dùng suốt đời chứ không chỉ trong thi đấu.

Thứ hai là quan sát nổi tiếng của ông về *tìm kiếm nhị phân*: ai cũng nghĩ mình viết đúng, còn thực tế thì rất nhiều lập trình viên chuyên nghiệp viết sai khi được yêu cầu viết từ đầu không tra cứu. Bài học không phải “tìm kiếm nhị phân khó” mà là: một thuật toán *đơn giản về ý tưởng* vẫn có thể rất khó viết đúng, nên phần cài đặt xứng đáng được luyện riêng như một kỹ năng độc lập.

Thứ ba là kỷ luật *đo trước khi tối ưu*. Trong bài viết cùng McIlroy về việc kỹ thuật hoá hàm sắp xếp của thư viện C[10], hai người mô tả cách một thuật toán sách vở phải chỉnh sửa nhiều lần mới sống được trong môi trường thật — với dữ liệu thù địch, với mảng đã sắp sẵn, với nhiều phần tử bằng nhau. Đây là ví dụ mẫu mực cho chương E/26.

Trường phái thi đấu: nhận diện bài toán quan trọng hơn thuộc thuật toán

Steven Skiena tổ chức *The Algorithm Design Manual*[11] thành hai nửa rất khác nhau: nửa đầu dạy kỹ thuật, nửa sau là một *danh mục bài toán* — bạn tra “tôi đang có bài toán trông như thế này” và sách chỉ cho bạn nó tên là gì, đã được giải chưa, và nên dùng thư viện nào. Cách tổ chức đó phản ánh một quan sát cốt lõi: trong thực tế, phần khó nhất thường không phải nghĩ ra thuật toán mà là *nhận ra bài toán của mình chính là bài toán kinh điển nào*.

Giới lập trình thi đấu đẩy quan sát này tới cùng. Bộ sách *Competitive Programming*[12] và *Guide to Competitive Programming*[13] tổ chức kiến thức thành các mẫu nhận dạng kèm bài luyện; các nguồn cộng đồng như cp-algorithms[14] và thư viện KACTL[2] cấp phần cài đặt đã kiểm chứng. Điểm chung trong lời khuyên luyện tập của những người ở nhóm dẫn đầu, lặp lại rất nhất quán

qua nhiều nguồn công khai, gồm bốn điểm: giải bài *hơi quá sức*, upsolving sau mỗi trận, tự cài lại thay vì đọc code người khác, và luyện có tính giờ.

I Kinh nghiệm người đi trước

Người / trường phái	Điều họ thực sự làm	Nguồn
Donald Knuth	Phân tích tới hằng số, không chỉ tới bậc tiệm cận. Bài tập xếp hạng 0–50 để người đọc chọn đúng vùng luyện. Thường tiền cho ai tìm ra lỗi trong sách của mình.	TAOCP[3], [4]
Edsger Dijkstra	Phát triển chương trình và chứng minh cùng lúc; viết bất biến trước khi viết vòng lặp; coi kiểm thử là công cụ phát hiện lỗi chứ không phải bằng chứng đúng.	<i>A Discipline of Programming</i> [6]
Tony Hoare	Đặt nền logic cho lập luận về chương trình (tiền điều kiện, hậu điều kiện); Quicksort ra đời từ việc tìm cách sắp xếp trước khi tra từ điển — ý tưởng đến từ ràng buộc thực tế.	[7], [15]
Robert Tarjan	Thiết kế cấu trúc dữ liệu song song với công cụ phân tích của nó: khẩu hao và hàm thể năng ra đời để phân tích chính những cấu trúc ông thiết kế.	[16], [17]
Jon Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng một thuật toán đơn giản là kỹ năng phải luyện riêng.	<i>Programming Pearls</i> [9]
Robert Sedgewick	Kết hợp phân tích toán học với thực nghiệm: cài đặt, đo, rồi đối chiếu với dự đoán lý thuyết; nếu lệch thì một trong hai phía sai.	[18], [19]
Steven Skiena	Xây danh mục bài toán để tra ngược từ “bài của tôi trông thế này”; nhấn mạnh nhận diện bài toán hơn là sáng tạo thuật toán mới.	<i>The Algorithm Design Manual</i> [11]
George Pólya	Bốn bước và một hệ thống câu hỏi để tự hỏi khi bí; dạy <i>quá trình</i> chứ không chỉ kết quả.	<i>How to Solve It</i> [20]
Alan Schoenfeld	Đo thực nghiệm quá trình giải bài; chỉ ra rằng tự giám sát là khác biệt lớn nhất giữa người giỏi và người kém.	[21]
Anders Ericsson	Luyện tập có chủ đích: mục tiêu hẹp, mức hơi quá sức, phản hồi ngay, sửa sai lập tức. Số giờ không tự sinh ra năng lực.	[1], [22]
Richard Hamming	Chọn bài toán quan trọng để làm; giữ thói quen suy nghĩ có hệ thống về việc mình đang làm gì và vì sao.	<i>The Art of Doing Science and Engineering</i> [23]
Peter Norvig	Không có đường tắt mười ngày; năng lực đến từ nhiều năm luyện có phản hồi, đọc code người khác, và làm dự án thật.	[24]
Giới lập trình thi đấu	Bốn thói quen lặp lại nhất quán: giải bài hơi quá sức, upsolving sau mỗi trận, tự cài lại thay vì chép, luyện có tính giờ; kèm một thư viện mẫu tự viết và đã kiểm chứng.	[2], [12], [13]

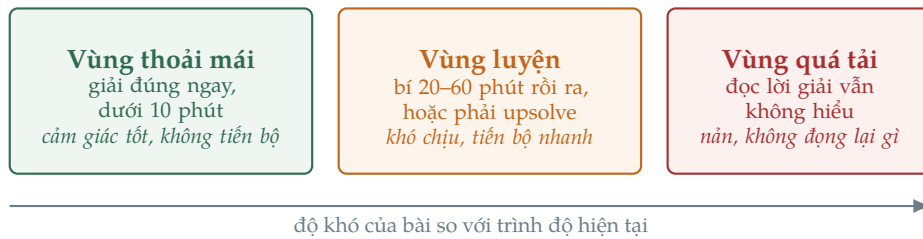
I 1.2 Luyện tập có chủ đích, dịch sang việc giải thuật

Nghiên cứu của Ericsson và cộng sự[1] đưa ra một kết luận đơn giản nhưng khó nuốt: số giờ luyện tập chỉ dự báo được năng lực khi việc luyện có bốn tính chất — mục tiêu hẹp và cụ thể, mức khó hơi vượt khả năng hiện tại, phản hồi gần như tức thì, và có sửa sai chứ không chỉ làm lại. Thiếu bất kỳ tính chất nào thì thời gian bỏ ra vẫn trôi, nhưng năng lực đứng yên.

Dịch sang việc luyện thuật toán, bốn tính chất trở thành bốn quy tắc rất cụ thể.

Mục tiêu hẹp. Không luyện kiểu “hôm nay học quy hoạch động”. Luyện kiểu “hôm nay làm năm bài quy hoạch động mà trạng thái là hai chiều, và với mỗi bài tôi phải viết ra định nghĩa trạng thái trước khi gõ code”. Mục tiêu hẹp cho phép biết mình có đạt hay không.

Vùng khó hơi vượt. Đây là quy tắc bị vi phạm nhiều nhất. Giải bài dễ cho cảm giác tốt — giải nhanh, đúng ngay, tự tin. Nhưng nếu bạn giải đúng ngay trong mười phút thì bài đó không dạy bạn gì. Vùng luyện đúng là vùng mà bạn giải được khoảng một nửa số bài, và nửa còn lại phải vật lộn rồi mới ra hoặc phải upsolve. Hình 1.2 vẽ ba vùng.



Hình 1.2. Ba vùng độ khó. Sai lầm phổ biến nhất khi tự luyện là ở lì trong vùng thoải mái vì nó cho cảm giác thành công; sai lầm ngược lại là nhảy thẳng vào vùng quá tải vì nghĩ rằng làm bài khó thì tiến bộ nhanh. Vùng luyện nằm giữa: bạn giải được khoảng một nửa, và luôn hơi khó chịu.

Phản hồi ngay. Đây là lý do các trang chấm bài tự động có giá trị lớn hơn vẻ ngoài của chúng: bạn biết đúng hay sai trong vài giây. Nhưng phản hồi “sai ở test 12” vẫn còn nghèo; phản hồi tốt hơn là stress test tự viết (chương E/25), vì nó chỉ thẳng ra *ca dữ liệu* nào làm hỏng.

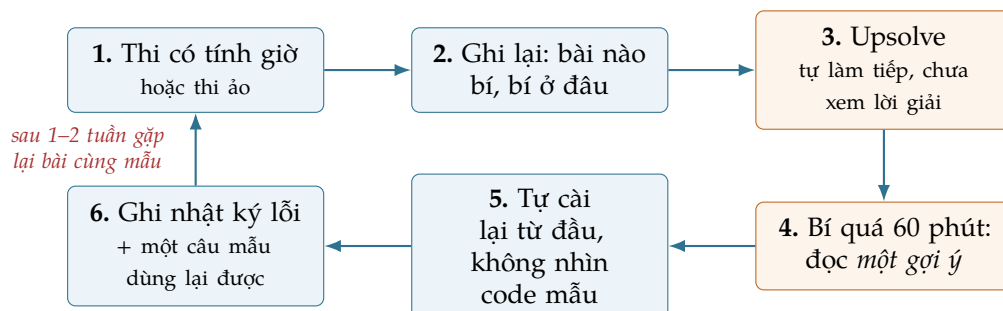
Sửa sai, không chỉ làm lại. Đây là chỗ nhét ký lỗi vào cuộc. Sau mỗi lần sai, ghi ba dòng: tôi đã hiểu sai điều gì, dấu hiệu nào trên đề lẽ ra đã gợi ý đúng, lần sau tôi sẽ kiểm tra gì. Sau vài tháng, nhét ký này sẽ cho bạn thấy một sự thật khó chịu và hữu ích: bạn sai đi sai lại đúng vài kiểu.

TRỰC GIÁC

Vì sao vùng thoải mái không tạo tiến bộ? Vì học xảy ra khi *dự đoán của bạn bị sai*. Giải một bài mà bạn biết chắc cách làm thì không có dự đoán nào bị vi phạm, nên không có gì để cập nhật. Cảm giác khó chịu khi bí chính là dấu hiệu của việc mô hình trong đầu bạn đang được sửa. Đó là lý do luyện đúng cách gần như luôn thấy không thoải mái, và vì sao cảm giác “buổi luyện hôm nay trôi chảy quá” thường là tin xấu.

1.3 Vòng upsolving — cỗ máy tiến bộ của dân thi đấu

Người mới thường nghĩ tiến bộ đến từ việc dự thi. Thực tế thì kỳ thi chỉ là phép đo; phần tạo ra tiến bộ nằm ở việc bạn làm gì trong vài ngày sau đó. Vòng lặp mà giới thi đấu dùng gần như thống nhất được vẽ ở hình 1.3.



Hình 1.3. Vòng upsolving. Hai ô tô hổ phách là phần tạo ra phần lớn tiến bộ, và cũng là phần hay bị cắt nhất vì nó không vui. Bước 5 — tự cài lại thay vì chép — là bước phân biệt “hiểu lời giải” với “làm được”; rất nhiều người tưởng mình đã học xong sau bước 4.

Cơ chế đằng sau vòng này khá rõ. Trong lúc thi, bạn ở trạng thái sức ép và đã tự mình đào rất sâu vào bài — tức là đã nạp đủ ngữ cảnh. Ngay sau đó, mỗi mẫu thông tin mới (một gợi ý, một ý tưởng) sẽ dính vào đúng chỗ trong đầu bạn, vì bạn biết chính xác nó lắp vào lỗ hổng nào. Đọc lời giải khi *chưa* vật lộn thì không có lỗ hổng nào để lắp, và thông tin trôi qua.

Một biến thể đáng dùng khi không có kỳ thi nào: *thi ảo*. Lấy một đề cũ, đặt đồng hồ đúng thời lượng thật, và làm nghiêm túc. Điểm quan trọng là tính giờ — vì kỹ năng phân bổ thời gian và kỹ năng chịu sức ép chỉ luyện được khi có sức ép.

CẠM BẤY

Cạm bẫy phổ biến nhất trong upsolving: đọc lời giải, gật gù “à hoá ra thế, dễ mà”, rồi chuyển bài. Cảm giác hiểu khi đọc là *cảm giác quen thuộc*, không phải năng lực tái tạo. Bài kiểm tra duy nhất đáng tin: đóng lời giải lại và tự cài từ trang trắng. Rất nhiều người phát hiện ra mình không làm được ở đúng bước này — và đó là thông tin quý, không phải điều đáng xấu hổ.

1.4 Đọc sách thế nào cho có tác dụng

Sách thuật toán bị đọc sai cách nhiều hơn bất kỳ loại sách kỹ thuật nào. Có ba cách đọc, và chỉ một cách tạo ra năng lực.

Đọc như tiểu thuyết — đọc từ đầu tới cuối, thấy hiểu. Cách này gần như không đọng lại gì sau ba tuần, vì không có lần nào bạn phải tự truy xuất kiến thức từ trí nhớ.

Đọc như từ điển — chỉ mở khi cần tra. Cách này hợp với nửa sau sách của Skiena hoặc với cp-algorithms, nhưng không xây được nền.

Đọc như phòng tập — đọc một mục, gấp sách, tự viết lại ý chính, rồi làm bài tập của mục đó *trước* khi đọc mục sau. Đây là cách duy nhất tạo ra năng lực, và nó chậm hơn hẳn: một chương có thể mất cả tuần. Knuth thiết kế TAOCP theo đúng giả định này — bài tập là một phần của nội dung, không phải phụ lục.

Bảng dưới đây là bản đồ sách: quyển nào cho vai trò nào. Đừng đọc song song quá hai quyển; chọn một quyển làm trục chính và dùng phần còn lại để tra.

Sách	Vai trò	Dùng khi nào
Pólya, <i>How to Solve It</i> [20]	Quy trình nghĩ	Đọc trước tất cả; mỏng, đọc trong một buổi
Zeitz[25], Engel[26]	Heuristic giải toán	Khi muốn luyện tư duy chứ không luyện thuật toán
Skiena[11]	Nhận diện bài toán	Trực chính tốt nhất cho người tự học; nửa sau là danh mục tra
CLRS[27]	Tham chiếu chuẩn	Tra định nghĩa, chứng minh chuẩn; <i>không</i> nên đọc tuần tự
Kleinberg–Tardos[28]	Dạy cách thiết kế	Khi muốn học cách <i>nghĩ ra</i> thuật toán, đặc biệt tham lam và luồng
Erickson[29]	Đệ quy và chứng minh	Miễn phí; phần quay lui và quy hoạch động rất tốt
Sedgewick–Wayne[18]	Cài đặt và đo	Khi muốn code chạy được, có thực nghiệm
Bentley[9]	Kỹ thuật thực dụng	Đọc rải rác, mỗi lần một chương ngắn
Laaksonen[13], Halim[12]	Thi đấu	Trực chính nếu mục tiêu là contest
Dijkstra[6], Gries[8]	Tính đúng	Khi bạn liên tục viết code gần đúng
Knuth[3]	Phân tích sâu	Tra một chủ đề cụ thể; không đọc tuần tự trừ khi có nhiều năm
Graham–Knuth–Patashnik[30]	Toán rời rạc	Khi phân tích độ phức tạp đệ quy tổng và truy hồi khó
Cormen <i>et al.</i> , Garey–Johnson[31]	Độ khó	Khi nghi ngờ bài của mình là NP-khó
McDowell[32]	Phỏng vấn	Bốn tới sáu tuần trước phỏng vấn, không sớm hơn

1.5 Những gì không bắt buộc được, và những gì bắt buộc được ngay

Trung thực mà nói, có những thứ trong danh sách trên bạn không nên cố bắt buộc. Knuth dành hàng chục năm cho một bộ sách; kiểu đầu tư đó chỉ hợp lý với một người có đúng mục tiêu của ông. Dijkstra làm việc phần lớn bằng bút và giấy, gần như không dùng máy tính — cách đó phù hợp với thời của ông và với tính chất công việc của ông, không phù hợp với người cần giao sản phẩm hàng tuần. Một số kỳ thủ lập trình bắt đầu từ rất sớm và luyện với cường độ mà người đi làm không thể có; so sánh bản thân với họ theo thang thời gian là cách nhanh nhất để nản vô ích.

Nhưng có sáu thứ bắt buộc được ngay trong tuần này, và chúng chiếm phần lớn giá trị:

1. Trước mỗi vòng lặp, viết một dòng bất biến bằng lời (Dijkstra).
2. Trước khi cài, ước lượng phong bì: n bao nhiêu, mỗi thao tác bao nhiêu, tổng cỡ bao nhiêu (Bentley).
3. Chọn bài ở vùng bạn giải được khoảng một nửa, không dễ hơn (Ericsson).
4. Sau mỗi trận hoặc mỗi buổi luyện, upsolve ít nhất một bài tới nơi tới chốn, tự cài lại từ trang trắng.
5. Giữ nhật ký lỗi ba dòng cho mỗi lần sai; đọc lại mỗi tháng.
6. Sau mỗi bài giải xong, viết một câu về ý tưởng cốt lõi không nhắc tới đề bài (Pólya, bước 4).

Sáu việc này cộng lại tốn chừng mười phút mỗi buổi luyện. Điểm chung của chúng: cả sáu đều là cách *đóng vòng phản hồi* — đúng thứ mà hình 1.1 nói là mẫu số chung của cả bốn trường phái.

1.6 Gốc rễ: vì sao ta biết những điều này về luyện tập

Những lời khuyên ở trên nghe như kinh nghiệm dân gian, nhưng phần lớn chúng có nguồn thực nghiệm, và nguồn ấy không đến từ ngành máy tính mà từ nghiên cứu về cờ vua.

Năm 1946, Adriaan de Groot — vừa là nhà tâm lý học vừa là kỳ thủ — đặt các kỳ thủ mạnh và yếu trước cùng một thế cờ và yêu cầu họ nói to suy nghĩ. Kết quả trái với kỳ vọng: kiện tướng *không* tính sâu hơn người trung bình bao nhiêu. Cái họ hơn là ngay từ vài giây đầu, họ đã nhìn ra một nhóm nhỏ nước đi đáng xét, trong khi người yếu hơn phải rà nhiều nước vô nghĩa. Nói cách khác, ưu thế nằm ở *nhận dạng*, không ở sức tính.

Đến 1973, William Chase và Herbert Simon — chính Simon của *Human Problem Solving* nhắc ở chương A/01 — làm thí nghiệm quyết định. Họ cho kỳ thủ nhìn một thế cờ trong vài giây rồi dựng lại. Kiện tướng dựng lại gần như hoàn hảo, người mới thì không. Nhưng khi các quân được đặt *ngẫu nhiên* thay vì theo thế cờ thật, ưu thế biến mất gần hết. Kết luận: chuyên gia không có trí nhớ tốt hơn; họ nhìn thấy *đơn vị lớn hơn* — những cụm quân có nghĩa, gọi là chunk. Trí nhớ của họ chỉ chứa vài chục đơn vị như mọi người, nhưng mỗi đơn vị đóng gói nhiều thông tin hơn.

Hai kết quả đó giải thích trực tiếp vì sao chương A/01 nhấn mạnh việc “viết một câu về ý tưởng cốt lõi không nhắc tới đề bài”. Câu đó chính là một chunk đang được tạo ra. Người nói “bài này dùng hai con trỏ” và người nói “khi cả hai đầu đều đơn điệu thì quét từ hai phía” đang lưu hai thứ rất khác nhau: một cái tên, và một điều kiện kích hoạt. Chỉ cái thứ hai mới tự nhận ra chính nó khi gặp bài mới.

Nhánh nghiên cứu thứ hai bắt đầu từ Benjamin Bloom, người phỏng vấn những cá nhân đạt đỉnh cao trong nhiều lĩnh vực và thấy một mẫu chung: nhiều năm luyện có hướng dẫn, tăng dần độ khó, với người dạy đóng vai trò cấp phản hồi. Ericsson và cộng sự hình thức hoá mẫu đó thành khái niệm luyện tập có chủ đích[1], [22]. Cần nói thẳng phần bị tranh cãi: con số “10 000 giờ” là cách phổ biến hoá của người khác, không phải kết luận của nghiên cứu, và mức độ mà luyện tập giải thích được khác biệt năng lực vẫn còn đang tranh luận trong giới nghiên cứu. Phần lỗi thì vững: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức hiệu quả hơn hẳn lặp lại thụ động — và đó là toàn bộ phần ta dùng.

Còn bản thân môi trường luyện thuật toán hiện đại thì trẻ hơn nhiều. Các kỳ thi lập trình đại học bắt đầu từ thập niên 1970 và lớn dần thành ICPC; kỳ thi tin học quốc tế dành cho học sinh phổ thông (IOI) bắt đầu năm 1989; các nền tảng chấm bài trực tuyến xuất hiện từ cuối thập niên 1990 và bùng nổ trong thập niên 2010. Điều mà những nền tảng này thay đổi không phải nội dung kiến thức mà là *độ trễ phản hồi*: từ vài tuần chờ thầy chấm xuống còn vài giây. Nếu tin vào Ericsson, thì đó chính là biến số quan trọng nhất, và nó giải thích vì sao trình độ trung bình của người luyện nghiêm túc ngày nay cao hơn hẳn hai mươi năm trước.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — de Groot	Muốn biết kiện tướng cờ nghĩ khác người thường ở đâu.	Kiến tướng không tính sâu hơn; họ nhận dạng nhanh hơn. Ưu thế nằm ở nhận dạng chứ không ở sức tính.
1973 — Chase & Simon	Kiểm chứng: đó là trí nhớ tốt hơn hay cách tổ chức tri thức khác?	Ưu thế biến mất với thế cờ ngẫu nhiên ⇒ chuyên gia lưu các <i>khối có nghĩa</i> . Giải thích vì sao phải phát biểu ý tưởng ở dạng tổng quát.
1985 — Bloom	Phỏng vấn người đạt đỉnh cao ở nhiều ngành để tìm mẫu chung.	Nhiều năm luyện có hướng dẫn và tăng dần độ khó; vai trò của người cấp phản hồi.
1993 — Ericsson và cộng sự	Vì sao cùng số giờ mà kết quả khác nhau.	Bốn điều kiện của luyện tập có chủ đích. Phần “10 000 giờ” là diễn giải của người khác và còn tranh cãi.
1989 và sau đó — IOI, chấm bài trực tuyến	Muốn đánh giá được kỹ năng lập trình ở quy mô lớn.	Độ trễ phản hồi tụt từ vài tuần xuống vài giây — thay đổi lớn nhất với người tự học trong 30 năm qua.

1.7 Liên hệ ngang: nghề nào cũng phải giải bài toán luyện tập này

Nếu bốn điều kiện của luyện tập có chủ đích là đúng, thì mọi nghề đòi hỏi kỹ năng cao đều phải tự phát minh ra cách thoả chúng. Và quả thật là vậy — nhìn sang các nghề khác, bạn sẽ thấy những cơ chế trùng khớp đến bất ngờ với upsolving và nhật ký lỗi, chỉ khác tên gọi. Hai chỗ đáng học nhất là y khoa và hàng không, vì họ đã *thể chế hoá* việc học từ lỗi thay vì phó mặc cho ý chí cá nhân.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Âm nhạc	Tập chậm từng đoạn khó, tách tay, dùng máy đếm nhịp tăng dần tốc độ	Giống: cô lập đúng phần yếu thay vì chơi lại cả bản — tương đương việc luyện riêng một dạng bài. Khác: nhạc công có phản hồi tức thì bằng tai, còn bạn phải tự tạo phản hồi bằng chấm bài và stress test.
Thể thao	Chu kỳ hoá: xen kẽ giai đoạn tăng khối lượng, tăng cường độ và phục hồi	Giống: cường độ cao liên tục không bền, cần xen kẽ. Gợi ý cho lộ trình ở E/28: xen tuần thi đấu với tuần học kiến thức mới. Khác: giải thuật không có giới hạn sinh lý, giới hạn là sự chú ý và động lực.
Y khoa	Hội chẩn tử vong và biến chứng (M&M): mỗi ca xấu được mổ xẻ công khai, không quy tội cá nhân	Đây chính là nhật ký lỗi được thể chế hoá, và nó hiệu quả hơn nhật ký cá nhân vì bài học được chia sẻ. Bắt buộc được: viết công khai bài phân tích lỗi của mình.
Hàng không	Bảng kiểm (checklist), báo cáo sự cố phi trường phạt, huấn luyện trên buồng lái mô phỏng	Bảng kiểm ứng với danh sách trường hợp biên trước khi nộp bài (E/25). Buồng lái mô phỏng ứng với thi ảo: rủi ro bằng không, lặp lại được, có thể cố tình tạo tình huống xấu.
Quân sự	Rà soát sau hành động (after-action review): điều dự định, điều đã xảy ra, vì sao khác, lần sau làm gì	Bốn câu này gần như trùng khít với bước 4 của Pólya và nội dung nhật ký lỗi. Điểm mạnh: nó là quy trình bắt buộc, không phụ thuộc tâm trạng.
Nghiên cứu khoa học	Số thí nghiệm, tái lập kết quả, bình duyệt	Số thí nghiệm chính là nhật ký lỗi. Bình duyệt là phản hồi từ người ngoài — thứ mà người tự học thiếu nhất; thay thế gần nhất là nhờ người khác đọc code hoặc viết editorial công khai.
Học máy	Vòng huấn luyện: mất mát trên tập kiểm tra, dừng sớm, học theo chương trình (curriculum learning)	Rất giống về cấu trúc: mẫu quá dễ cho gradient nhỏ, mẫu quá khó cho gradient nhiều; vùng học hiệu quả nằm giữa — đúng hình 1.2. Khác: mô hình không tự chọn được bài, còn bạn thì có.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thói quen của họ không phải sở thích cá nhân mà là cách đóng vòng phản hồi, và vòng phản hồi thì chỉ có vài cách đóng. Bốn trường phái ở hình 1.1 khác nhau ở thước đo — toán, logic, đồng hồ, số bài giải được — nhưng ai cũng phải có một thước đo khách quan và phải sửa cách làm theo kết quả đo. Người luyện không có thước đo nào là người duy nhất không tiến bộ.
- A2. Vì đọc tạo ra *cảm giác quen thuộc*, còn làm bài tập tạo ra *khả năng truy xuất*. Hai thứ đó khác nhau: bạn có thể thấy mọi dòng chứng minh đều hiển nhiên khi đọc mà vẫn không tái tạo được nó khi gấp sách. Bài tập ép bạn truy xuất, và chính lúc truy xuất khó khăn thì trí nhớ mới được củng cố. Cách xếp hạng 0–50 của Knuth còn giúp bạn chọn đúng vùng khó thay vì làm hết mọi bài.
- A3. Nó đổi thứ tự công việc. Thay vì gõ code rồi tìm ca dữ liệu làm nó sai, bạn viết ra điều kiện cần giữ đúng *trước*, và code trở thành cách thực hiện điều kiện đó. Cụ thể nhất là một dòng bắt biến trước mỗi vòng lặp: khi có nó, các câu hỏi “bắt đầu từ 0 hay 1”, “chạy tới n hay n-1”, “khởi tạo bằng 0 hay $-\infty$ ” đều được suy ra thay vì đoán rồi thử.

- A4. Vì “nhiều bài” thường có nghĩa là nhiều bài *vừa sức*, mà vừa sức thì không tạo ra sai lệch dự đoán để học. Sau vài tháng bạn thành thạo hơn ở những gì đã biết và không mở rộng được gì mới. Điều kiện Ericsson đòi hỏi là mức hơi quá sức cộng phản hồi cộng sửa sai; giải ít bài hơn nhưng chọn ở vùng luyện, có upsolve và có nhật ký lỗi thì thoả cả ba.
- A5. Vì trong lúc thi bạn đã tự đào rất sâu vào bài, tức là đã tạo sẵn những “lỗ hổng” rất cụ thể trong hiểu biết. Ngay sau đó, một gợi ý nhỏ sẽ rơi đúng vào lỗ hổng và dính lại. Nếu đọc lời giải khi chưa vật lộn thì không có lỗ hổng nào, thông tin trôi qua và bạn chỉ thu được cảm giác đã hiểu. Đó cũng là lý do bước “tự cài lại từ trang trắng” không được bỏ.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn học chơi đàn. Có bốn kiểu thầy: người bắt bạn đếm nhịp thật chính xác, người bắt bạn hiểu vì sao hợp âm này đi với hợp âm kia, người bắt bạn thu âm lại rồi nghe xem chỗ nào sai, và người bắt bạn lên sân khấu đúng giờ. Bốn kiểu ấy tương ứng với bốn nhóm người trong chương này. Điều họ giống nhau là ai cũng có một cách để *biết mình đang sai ở đâu* — đếm nhịp, lý thuyết, bản thu, khán giả. Người tập đàn một mình trong phòng, không thu âm, không ai nghe, thì tập bao nhiêu năm cũng chỉ giỏi lên chút ít, vì không có gì chỉ cho họ chỗ sai.

Việc luyện thuật toán y hệt. Bạn cần một cái “bản thu”: chấm bài tự động, stress test, đồng hồ bấm giờ, hay đơn giản là quyển sổ ghi mỗi lần bạn sai. Và bạn cần chọn bài ở mức hơi khó — dễ quá thì như tập lại bài đã thuộc, khó quá thì như ngồi trước bản nhạc không đọc nổi nốt nào. Cuối cùng, sau mỗi lần tập, hãy quay lại đúng chỗ bạn vấp và tự chơi lại từ đầu, chứ đừng nghe người khác chơi rồi tưởng mình chơi được.

Một câu: Người giỏi không luyện nhiều hơn bạn nhiều đến thế; họ luyện ở mức khó hơn một chút và luôn biết mình vừa sai ở đâu.

Chỗ không rút gọn được: Có phần thật sự không rút gọn được: thời gian. Vòng phản hồi tốt làm mỗi giờ luyện đáng giá hơn, nhưng không có cách nào bỏ qua vài trăm giờ đầu tiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn điều kiện của luyện tập có chủ đích	Mục tiêu hẹp; mức hơi quá sức; phản hồi gần như tức thì; có sửa sai chứ không chỉ làm lại. Thiếu một điều thì thời gian trôi mà năng lực đứng.
••• Vùng luyện đúng	Vùng bạn giải được khoảng một nửa số bài; luôn hơi khó chịu. Dễ quá thì không có dự đoán nào bị sai để mà học.
••• Vòng upsolving	Thi tính giờ → ghi chỗ bí → tự làm tiếp → chỉ đọc <i>một</i> gợi ý khi bí quá 60 phút → tự cài lại từ trang trắng → ghi nhật ký lỗi.
••• Kiểm tra “đã học được” đáng tin duy nhất	Đóng lời giải, tự cài từ trang trắng. Cảm giác hiểu khi đọc chỉ là cảm giác quen thuộc.
••• Câu của Dijkstra về kiểm thử	Kiểm thử cho thấy sự hiện diện của lỗi, không bao giờ cho thấy sự vắng mặt của lỗi — nên tính đúng phải được xây vào lúc viết, không thêm vào lúc kiểm.
•• Câu Knuth bị trích sai	“Tối ưu hoá sớm là gốc rễ mọi tội lỗi” — nguyên ý là bỏ qua cái thiện nhỏ trong phần lớn thời gian, nhưng phải đo để tìm ra phần thực sự quan trọng và tối ưu ở đó.
•• Đóng góp phương pháp của Tarjan	Thiết kế cấu trúc dữ liệu cùng với công cụ phân tích của nó; khẩu hao và hàm thế năng ra đời từ nhu cầu đó.
•• Ba thói quen của Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng thuật toán đơn giản là kỹ năng phải luyện riêng.
•• Vì sao Skiena tổ chức sách thành danh mục	Vì trong thực tế, phần khó nhất thường là nhận ra bài của mình chính là bài kinh điển nào, chứ không phải nghĩ ra thuật toán mới.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba cách đọc sách và cách nào có tác dụng	Đọc như tiểu thuyết (trôi), như từ điển (tra được, không xây nền), như phòng tập (gấp sách, viết lại, làm bài tập trước khi sang mục sau) — chỉ cách thứ ba tạo năng lực.
●● Nội dung ba dòng của nhật ký lỗi	Tôi hiểu sai điều gì; dấu hiệu nào lẽ ra đã gợi ý đúng; lần sau tôi sẽ kiểm tra gì.
● Thi ảo	Lấy đề cũ, đặt đồng hồ đúng thời lượng thật, làm nghiêm túc — dùng khi không có kỳ thi thật, để luyện phân bổ thời gian dưới sức ép.
● Bốn trường phái và thước đo của họ	Phân tích (toán), chứng minh (logic), kỹ thuật (đồng hồ trên máy thật), thi đấu (số bài giải được có tính giờ).

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi đi làm, mỗi ngày chỉ có một giờ. Nên phân bổ thế nào cho hiệu quả nhất?

| Một cách phân bổ dùng được: 10 phút ôn lại (đọc nhật ký lỗi và bảng active recall của chương đang học), 40 phút giải một bài ở vùng luyện có bầm giờ, 10 phút cho bước 4 — viết một câu về ý tưởng cốt lõi và ghi nhật ký nếu sai. Điều quan trọng hơn tỉ lệ là tính đều đặn: một giờ mỗi ngày trong sáu tháng hơn hẳn tám giờ mỗi cuối tuần, vì trí nhớ củng cố theo số lần truy xuất chứ không theo tổng thời gian.

2. Tôi nên đọc hết CLRS không?

| Gần như chắc chắn là không, trừ khi bạn đang học một khoá có giáo trình đó. CLRS được thiết kế làm tham chiếu chuẩn: định nghĩa chính xác, chứng minh đầy đủ, bao phủ rộng. Đọc tuần tự thì rất chậm và bạn sẽ dừng ở khoảng chương 8. Cách dùng đúng: lấy một quyển khác làm trục chính (Skiena nếu tự học, Laaksonen nếu thi đấu), và mở CLRS khi cần một chứng minh chuẩn hoặc một chi tiết mà trục chính nói lướt.

3. Đọc code của người giỏi có ích không, hay chỉ nên tự viết?

| Cả hai, nhưng đúng thứ tự. Đọc code trước khi tự vật lộn thì bạn chép được kiểu viết mà không hiểu vì sao; đọc sau khi đã tự cài xong thì rất có giá trị, vì bạn thấy ngay chỗ họ ngắn hơn, ít trường hợp riêng hơn, hoặc chọn biểu diễn dữ liệu khôn hơn. Một bài tập tốt: cài xong rồi so với thư viện tham chiếu như KACTL[2] và tự hỏi vì sao họ chọn cách đó.

4. Xây thư viện mẫu cho riêng mình có đáng không, khi đã có thư viện công khai?

| Đáng, vì lợi ích chính không nằm ở đoạn code mà ở quá trình viết. Khi tự cài DSU hay segment tree tới mức đúng và gọn, bạn buộc phải hiểu từng chi tiết, và về sau bạn sửa được nó cho biến thể lạ — điều không làm được với code chép về. Nguyên tắc: mỗi đoạn trong thư viện riêng của bạn phải do bạn viết và đã qua stress test.

5. Làm sao biết mình đang ở vùng luyện chứ không phải vùng thoải mái?

| Bằng tỉ lệ giải được. Nếu bạn giải đúng ngay gần như mọi bài trong dưới 15 phút, bạn đang ở vùng thoải mái — nâng độ khó. Nếu bạn không giải được gì trong hai buổi liên tiếp và đọc lời giải vẫn thấy mù mờ, bạn đang ở vùng quá tải — hạ xuống. Điểm cân bằng là khoảng một nửa số bài giải được sau khi vật lộn, nửa còn lại phải upsolve.

6. Nghiên cứu về luyện tập có chủ đích có bị thổi phồng không?

| Có phần. Kết luận mạnh kiểu “chỉ cần 10 000 giờ là thành chuyên gia” là cách diễn giải phổ biến hoá, và bị tranh luận nhiều trong giới nghiên cứu; các yếu tố khác như điểm xuất phát và điều kiện học tập đều có vai trò. Nhưng phần lõi thì vững và cũng là phần ta dùng ở đây: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức thì hiệu quả hơn hẳn luyện lặp lại thụ động. Ta không cần phần bị tranh cãi để dùng được phần lõi.

7. Chuyên gia cờ vua nhớ thể cờ giỏi hơn người thường — điều đó liên quan gì tới thuật toán?

| Liên quan trực tiếp, và đây là một trong những kết quả kinh điển của tâm lý học nhận thức: ưu thế trí nhớ đồ biến mất khi các quân được đặt ngẫu nhiên. Nghĩa là chuyên gia không nhớ tốt hơn, họ *nhìn thấy đơn vị lớn hơn* — những cụm có nghĩa. Với thuật toán cũng vậy: người có kinh nghiệm không nhớ nhiều dòng code hơn, họ nhìn một đề bài và thấy “đây là bài ghép đôi trá hình”. Xây các khối có nghĩa đó chính là việc mà bước 4 của Pólya và câu “phát biểu ý tưởng không nhắc đề bài” đang làm.

8. Tôi hay bỏ dở lộ trình sau vài tuần. Có mẹo nào không?

| Ba điều chỉnh có hiệu quả rõ. Thứ nhất, hạ mục tiêu hằng ngày xuống mức bạn chắc chắn làm được kể cả ngày bận — một bài, hoặc thậm chí 20 phút; tính liên tục quan trọng hơn khối lượng. Thứ hai, làm cho tiến bộ nhìn thấy được: một bảng đánh dấu ngày, hoặc danh sách bài đã upsolve. Thứ ba, ghép việc luyện vào một khung giờ cố định thay vì “khi nào rảnh”. Chương E/28 có lộ trình cụ thể theo tuần đã tính tới ba điều này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tự viết lại tìm kiếm nhị phân từ trang trắng, không tra cứu	Bài tập của Bentley[9]	Viết bất biến trước; sau đó stress test với mảng rỗng, một phần tử, nhiều phần tử trùng nhau.
● Đọc mục 1 của <i>How to Solve It</i> và áp bốn bước vào một bài bạn đang bí	[20]	Mục tiêu là tập nhận ra mình đang ở bước nào, đặc biệt phát hiện lúc bạn nhảy cóc sang bước 3.
● Lập nhật ký lỗi và ghi ba dòng cho năm lần sai gần nhất	Bài tập tự tổ chức	Sau năm mục, tự phân loại: sai do hiểu đề, sai do ý tưởng, sai do cài đặt. Tỷ lệ này quyết định bạn nên luyện gì.
●● Chọn một đề contest cũ và làm thi ảo đúng thời lượng	Codeforces / AtCoder / CSES[33]	Ghi lại mốc thời gian mỗi lần đổi bài; sau đó đối chiếu với hình phân bố thời gian ở chương A/01.
●● Upsolve một bài trong đề vừa làm, tự cài lại từ trang trắng	Cùng đề trên	Bước 5 của vòng upsolving; đây là bài tập quan trọng nhất trong cả danh sách này.
●● Cài DSU và segment tree cho thư viện riêng, có stress test	[2] để đối chiếu	So sánh code của bạn với thư viện tham chiếu và giải thích từng khác biệt — đừng chép, hãy giải thích.
●● Chọn một chương của Skiena, làm hết bài tập trước khi đọc chương sau	[11]	Luyện cách đọc “như phòng tập”; ghi lại thời gian thật để biết tốc độ đọc bền vững của bạn.
●●● Viết editorial cho một bài bạn vừa giải, như thể dạy người khác	Bài tập kiểu Feynman	Chỗ bạn viết không trôi chảy là chỗ bạn chưa hiểu; đó là mục đích duy nhất của bài tập này.
●●● Lấy một thuật toán bạn “đã biết” và tự chứng minh tính đúng bằng bất biến	Chương A/04	Chọn Dijkstra hoặc thuật toán tham lam xếp lịch; nếu không viết được chứng minh, bạn mới chỉ nhớ chứ chưa hiểu.
●●● Thiết kế lộ trình 12 tuần cho riêng bạn, có mục tiêu đo được mỗi tuần	Chương E/28 làm mẫu	Mỗi tuần phải có một tiêu chí kiểm tra khách quan, không phải “học xong chương X”.

Kết chương và đọc tiếp

Chương này rút gọn được thành một câu: những người giỏi nhất khác nhau ở thước đo họ chọn, nhưng giống nhau ở chỗ ai cũng có một thước đo và ai cũng sửa cách làm theo kết quả đo. Bạn không sao chép được hoàn cảnh của họ, nhưng sáu thói quen ở mục cuối thì sao chép được ngay, và chúng chính là phần tạo ra phần lớn giá trị.

Còn bỏ ngỏ hai câu hỏi thực dụng mà chương này liên tục giả định đã có câu trả lời: thước đo “đủ nhanh” là gì, và thước đo “chắc chắn đúng” là gì. Hai chương tiếp theo cấp đúng hai thước đo đó — A/03 cho chi phí, A/04 cho tính đúng. Sau đó, E/28 sẽ biến toàn bộ chương này thành một lịch luyện cụ thể theo tuần.

Nguồn

- [1] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [2] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [3] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [4] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [5] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [6] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [7] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [8] D. Gries, *The Science of Programming*. Springer, 1981.
- [9] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [10] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [11] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [12] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [13] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [14] e-maxx / cp-algorithms contributors. “cp-algorithms: Algorithms for Competitive Programming.” (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [15] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [16] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [17] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [18] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [19] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [20] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [21] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [22] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.
- [23] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.

- [24] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [25] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [26] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).